

DeepStochLog: Neural Stochastic Logic Programming

Thomas Winters^{*1}, Giuseppe Marra^{*1}, Robin Manhaeve¹, and Luc De Raedt^{1,2}

¹KU Leuven, Dept. of Computer Science; Leuven.AI, B-3000 Leuven, Belgium,
`{firstname.lastname}@kuleuven.be`

²AASS, Örebro University, Sweden

Abstract

Recent advances in neural symbolic learning, such as DeepProbLog, extend probabilistic logic programs with neural predicates. Like graphical models, these probabilistic logic programs define a probability distribution over possible worlds, for which inference is computationally hard. We propose DeepStochLog, an alternative neural symbolic framework based on stochastic definite clause grammars, a type of stochastic logic program, which defines a probability distribution over possible derivations. More specifically, we introduce neural grammar rules into stochastic definite clause grammars to create a framework that can be trained end-to-end. We show that inference and learning in neural stochastic logic programming scale much better than for neural probabilistic logic programs. Furthermore, the experimental evaluation shows that DeepStochLog achieves state-of-the-art results on challenging neural symbolic learning tasks.

1 Introduction

The integration of neural and symbolic learning methods is high on the research agenda. It is popular to use (variants of) logic programs to represent the available symbolic knowledge and use Prolog-like mechanisms to generate computation structures that can then be differentiated [23, 7, 31, 43, 5, 37, 36]. Several of these approaches also incorporate probability into these neural logic programming models, cf. [8]. There are several ways to incorporate probability into logic programs [9]. Most notably, one distinguishes probabilistic from stochastic logic programs (PLPs vs SLPs). The more popular PLPs are based on a possible worlds semantics (the so-called distribution semantics), which extends probabilistic graphical models, while the SLPs are based on stochastic grammars. The difference can also be described as a random graph vs a random walk model. So far, the emphasis in neurosymbolic computation has been on the PLP approach, especially [43, 23, 37, 40], with only Tensorlog [5] adopting the SLP semantics in an efficient but restricted Datalog or database setting that does not handle subsymbolic data such as images (see Section 7 for details).

To fill this gap, we introduce DeepStochLog, a neural stochastic logic programming approach. It incorporates ideas from DeepProbLog such as the neural predicate. The neural predicate encapsulates neural networks to cope, for instance, with subsymbolic data such as images. Without loss of generality, we base DeepStochLog on stochastic definite clause grammars (SDCGs) as this notation is not only easier to introduce and use, but also results in a sequence-based model. SDCGs are a kind of probabilistic unification-based grammar formalism [15]. However, SDCGs and SLPs are very closely related. SDCGs can be directly translated and executed as SLPs, and all the concepts we introduce for SDCGs can directly apply to SLPs as well. More specifically, the key contributions of this paper are: 1) the introduction of the neural stochastic logic programming framework DeepStochLog; 2) the introduction of inference and learning algorithms (through gradient

^{*}The authors contributed equally to this paper.

Note that this paper will be extended in the future.

descent) for DeepStochLog programs; and 3) experimental results that show that DeepStochLog obtains state-of-the-art results on a number of challenging tasks for neural-symbolic computation and that it is also several orders of magnitude faster than alternative approaches based on PLPs.

2 Stochastic DCGs

A context-free grammar (CFG) G is a 4-tuple (V, Σ, S, R) , with V the set of non-terminals, Σ the set of terminals, $S \in V$ the starting symbol and R a set of rewrite rules of the form $N \rightarrow S_1, \dots, S_k$ where N is a non-terminal, the S_i are either terminals or non-terminals. A probabilistic context-free grammar (PCFG) extends a CFG by adding probabilities to the rules R , i.e., the rules take the form $p_i :: N \rightarrow S_1, \dots, S_k$, where p_i is a probability. Furthermore, the sum of the probabilities of rules with the same non-terminal N on the left-hand side equals 1. We use list notation for sequences of terminals such as $[cat]$ and $[the, cat]$. Whereas CFGs define whether a sequence can be parsed, PCFGs define a probability distribution over possible parses. This allows for the most likely parse to be identified. An example PCFG, representing single digit additions, is shown on the left of Example 1.

Definite clause grammars (DCGs) are a well-known logic programming-based extension of CFGs [28]. DCGs can represent context-sensitive languages and are unification-based. They differ from CFGs in that logical atoms are used instead of the non-terminals. An *atom* $a(t_1, \dots, t_n)$ consists of a predicate a of arity n followed by n terms t_i . Terms are either constants, logical variables or structured terms of the form $f(t_1, \dots, t_k)$ with f a functor and t_j terms. The production rules are called definite clause grammar rules because they can be directly translated to a set of definite clauses (i.e., Horn clauses with exactly one positive literal) and can be executed as a Prolog program using SLD-resolution. The right hand side of DCG rules are also allowed to contain queries to Prolog predicates q_i between curly brackets $\{q_1(t_{1,1}, \dots, t_{1,m_1}), \dots, q_n(t_{n,1}, \dots, t_{n,m_n})\}$ to impose further constraints and perform additional computations during the inference process. These are to be considered atoms as well. DCGs use substitutions $\{V_1 = t_1, \dots, V_n = t_k\}$, which are sets of variable/term pairs, to unify atoms with heads from the rules. Applying a substitution to an atom a yields the atom $a\theta$ where all variables V_i have been replaced by their corresponding terms t_i . θ is a unifier of an atom s and an atom u if and only if $s\theta = u\theta$. For more information, see standard textbooks on logic programming such as [14, 38].

Stochastic definite clause grammars (SDCGs) extend DCGs by associating probabilities to the rules, just like how PCFGs extend CFGs [15]. As PCFGs, SDCGs require that the sum of the probabilities for the rules defining a single non-terminal predicate equals 1. SDCGs also correspond directly to stochastic logic programs (SLP) [6, 25, 26] which are well-known in the probabilistic (logic) programming community [9]. An example SDCG is shown in Example 1 on the right.

Example 1. A PCFG (left) and a similar SDCG(right) that constrains the result of the expression.

$0.5 :: E \rightarrow N$	$0.5 :: e(N) \rightarrow n(N)$
$0.5 :: E \rightarrow E, [+], N$	$0.5 :: e(N) \rightarrow e(N1), [+], n(N2), \{N \text{ is } N1 + N2\}$
$0.1 :: N \rightarrow [0] \quad \dots \quad 0.1 :: N \rightarrow [9]$	$0.1 :: n(0) \rightarrow [0] \quad \dots \quad 0.1 :: n(9) \rightarrow [9]$

The inference task in (S)DCGs consists of deriving a sequence of terminals from a goal (which often captures the starting symbol of the grammar). SLD-derivations are used for this. More formally, in an SLD-derivation for a DCG, a *goal* $g_1, \dots, g_n$ is a sequence where each g_i is either a logical atom (a non-terminal) or a list containing terminals and logical variables. An SLD-derivation is shown in Example 2 and uses several resolution steps. Applying *resolution* to a goal $g_1, \dots, g_n$ and a definite clause grammar rule $n \rightarrow t_1, \dots, t_k$ yields the goal $g_1\theta, \dots, g_{i-1}\theta, t_1\theta, \dots, t_k\theta, g_{i+1}\theta, \dots, g_n\theta$ provided that g_i is the leftmost atom in the goal (so $g_1, \dots, g_{i-1}$ are terminal symbols), θ is the unifier of g_i and n , i.e., $g_i\theta = n\theta$.¹ We write this as $g_1, \dots, g_n \vdash t_1\theta, \dots, t_k\theta, s_2\theta, \dots, s_n\theta$. A derivation

¹When a Prolog query q is the first non-terminal to occur in a goal during the derivation process, the query is

$d(S)$ is then the repeated application $G \vdash G_1\theta_1 \vdash G_2\theta_1\theta_2 \vdash \dots \vdash G_n\theta_1\theta_2\dots\theta_n$ of such resolution steps onto a goal G . We will write $G \vdash^* G_n$. Successful derivations of a goal end in a sequence T that consists only of terminal symbols, see Example 2 for an example. We will write that $d(S) = T$ and also say that $derives(S\theta, T)$ is true, with $\theta = \theta_1\theta_2\dots\theta_n$ the answer substitution. A successful derivation corresponds to a proof. The set of all possible proofs can be depicted using SLD-trees, see Figure 1a.

The probability $P(d(G))$ of a derivation $d(G)$ is the product of the probabilities $\prod p_i^{m_i}$ of the rules i used in the derivation with m_i the number of times the rule i was used. An important difference between the probability of a parse in a PCFG and a derivation in an SDCG is that there can be a loss of probability mass in the latter whenever a derivation fails. Derivations can fail when there are non-terminals in the goal that do not unify with the heads of any of the rules. This is due to unification and is different from (P)CFGs, where non-terminals can always be resolved using rules for that non-terminal. Non-terminating derivations can also lead to loss of probability mass. Observe that every G in this way induces a probability distribution P_G over possible derivations $d(G)$. The goal can consist of one or more atoms, but in general for parsing, this will typically be the starting symbol or atom of the grammar. This in turn lets us define the probability $P_G(derives(G, T))$ of a terminal sequence T relative to the goal G as $\sum_{d_i(G\theta)=T} P_G(d_i(G\theta))$, i.e. the sum of the probabilities of all derivations $G\theta$ for G that result in the terminal sequence T with answer substitution θ . In a similar way, this allows to define the probability of an answer substitution θ relative to a goal G and sequence t as $P_G(derives(G\theta, t\theta))$ where t could contain variables. For ease of notation, when the goal is clear, we shall omit the subscript G . Notice that if there are failing derivations, the total probability mass assigned to all sequences of terminals for a goal G may be strictly less than 1. This is discussed at length by [6]. It is possible to obtain normalized probabilities by calculating the normalization constant, but this is computationally expensive. We avoid this normalization in the present paper because in practice, the goal is often to find the most likely derivation $d_{max}(G, T) = \arg \max_{d(G)=T} P_G(d(G))$, non-normalized probabilities usually suffice. Notice that a SDCG defines a parametric probability distribution, where the parameters p of the distribution are the vector of probabilities of the rules. When we need to refer to these probabilities we write $P_G(derives(G, T); p)$.

Example 2 (Derivations). Consider the following successful derivation using the SDCG in Example 1 for the goal $G = [e(X)]$, the answer substitution $\theta = \{X/2\}$ and the terminal sequence $T = [2, +, 0]$.

$e(X) \vdash e(N1), [+], n(N2), \{X \text{ is } N1 + N2\}$	$\theta_1 = \{\}$	$p = 0.5$
$\vdash n(N1), [+], n(N2), \{X \text{ is } N1 + N2\}$	$\theta_2 = \{\}$	$\times 0.5$
$\vdash [2, +], n(N2), \{X \text{ is } 2 + N2\}$	$\theta_3 = \{N1/2\}$	$\times 0.1$
$\vdash [2, +, 0], 2 \text{ is } 2 + 0$	$\theta_4 = \{X/2, N2/0\}$	$\times 0.1$

3 DeepStochLog

DeepStochLog integrates neural networks and SDCGs by introducing neural definite clause grammars (NDCG). More formally, DeepStochLog allows for specifying an SDCG that additionally supports *neural definite clause grammar rules*, or neural rules for short. These are statements of the form:

$$nn(m, [I_1, \dots, I_m], [O_1, \dots, O_L], [D_1, \dots, D_L]) :: nt \rightarrow g_1, \dots, g_n$$

where nt is an atom, $g_1, \dots, g_n$ is a goal, and the $I_1 \dots I_m$ and $O_1, \dots, O_L$ are variables occurring in $g_1, \dots, g_n$ and nt . The D_i are unary predicates defining the domain of the output variables O_i . The nn declaration states that m is a neural network that takes the variables $I_1, \dots, I_m$ as input and

executed in Prolog possibly yielding an answer substitution θ such that $q\theta$ is true. For instance, in Example 1, there is the Prolog query $N \text{ is } N1 + N2$ which computes N as the sum of $N1$ and $N2$. In this paper, we assume that when such a query is called there is at most one answer substitution that is true. If there were more such substitutions, we would have to introduce a probability for such substitutions in the SDCG case, which unnecessarily complicates the semantics.